

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-ab54df30d6b25f848.jsonl`
- SHA-256: `1e6db5c559b83b7d0542a351f07ea6714b6238164021bf6775a11234435e6d0c`
- Source modified: `2026-06-13T19:15:36+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:13:53.218Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

```
FINDING: {"title":"int(cw['start']*fps) frame round-trip truncates to frame-1 for hundreds of frames at fps=30/29.97/25 (window-edge off-by-one)","severity":"low","file":"backend/pipeline/segmenters/fusion_hybrid.py:97,105","detail":"Windows carry start/end in seconds (trajectory_to_action_windows: `start = group[0][0]/fps`, tracknet_runner.py:300-301). _person_features reconstructs the frame bound via `start_f = int(cw['start']*fps)` (and end_f similarly). Because float `frame/fps*fps` can land just below the integer, `int(...)` truncation yields frame-1 for many frames: I measured 404 mismatches in [0,20000) at fps=30, 477 at 29.97, 1145 at 25.0 (e.g. frame 123 -> 122). Consequence is bounded and mostly benign: detections_per_frame is seeded one frame early (sampling phase shifts by one but still aligns 1-in-step with shuttle .frame), and the `shuttle_in = [p for p in points if start_f <= p.frame <= end_f]` filter may include/exclude exactly one boundary frame. Not corrupting, but it means the window bound is not the exact frame the substrate intended. The cv2 keyframe-seek tolerance (documented in the docstring) already dwarfs this, so low severity.","fix":"Use round() instead of int() for the seconds->frame conversion (`start_f = round(cw['start']*fps)`), or better, thread the integer frame indices from the windowing step through cw (e.g. persist start_frame/end_frame on the window dict in trajectory_to_action_windows) so no float round-trip is needed."}
```

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:55.897Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:13:56.461Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
```

```

3 Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4 TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5 do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6 shuttle-seeded window via two overridable seams the engine exposes:
7
8 - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9 GPU-free, computed from the trajectory we already have); and, only when the person cue
10 is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11 (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12 fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13 never imports torch / touches the GPU.
14 - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15 (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16 dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17 **Persisted candidates remain the full superset** regardless, so the offline
experiment
18 harness can re-score any variant.
19
20 Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21 nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22 and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23 segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24 experiment harness (EXPERIMENT_HARNESS.md), never silently.
25 """
26 from typing import Any, Dict, List, Optional, Tuple
27
28 from backend.pipeline.segmenters.base import segmenter_registry
29 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30 from backend.pipeline.detectors.base import DetectorRunner
31 from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33 from backend.pipeline.detectors import rally_gate
34 from backend.pipeline.detectors import fusion_features as ff
35
36
37 class FusionSegmenter(TrajectoryHybridSegmenter):
38     """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40     family = "fusion"
41     display_label = "Fusion"
42
43     def __init__(self, db: Any, config: Any):
44         super().__init__(db, config)
45         # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46         self._person: Optional[Any] = None
47
48     @property
49     def name(self) -> str:
50         return "fusion_hybrid"
51
52     @property
53     def description(self) -> str:
54         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                 "AI provider sets semantic boundaries.")

```

[illegible]

```

111                 idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

4. assistant (2026-06-13T19:13:59.705Z)

Let me look at the key code paths the finding references. Let me check
`trajectory\_to\_action\_windows` in tracknet_runner.py and the windowing in trajectory_hybrid.py.

5. user (2026-06-13T19:14:00.689Z)

```

270         """
271         if fps <= 0:
272             fps = 30.0
273         if frame_width <= 0:
274             frame_width = 1280.0
275
276         # Compute per-frame normalised velocity from the last *visible* point.
277         active = [] # (frame_idx, velocity)
278         prev: Optional[TrajectoryPoint] = None
279         for p in points:
280             if not p.visible:
281                 prev = None # break the trajectory; absent shuttle = not in flight
282                 continue
283             if prev is not None:
284                 dframes = p.frame - prev.frame
285                 if dframes > 0:
286                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
287                     velocity = (dist / frame_width) * (fps / dframes)
288                     if velocity > velocity_thresh:
289                         active.append((p.frame, velocity))
290             prev = p
291
292         if not active:
293             return []
294
295         max_gap_frames = int(merge_gap * fps)
296
297         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
298             vels = [v for _, v in group]
299             return {
300                 "start": group[0][0] / fps + chunk_start,
301                 "end": group[-1][0] / fps + chunk_start,
302                 "active_frame_count": len(group),
303                 "peak_velocity": max(vels),

```

```

304         "mean_velocity": sum(vels) / len(vels),
305         "track_id_count": 1, # single shuttle; kept for window-shape parity
306         "chunk_index": chunk_index,
307     }
308
309     windows = []
310     group = [active[0]]
311     for af in active[1:]:
312         if af[0] - group[-1][0] <= max_gap_frames:
313             group.append(af)
314         else:
315             windows.append(_finalize(group))
316             group = [af]
317     windows.append(_finalize(group))
318
319     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
320

```

6. user (2026-06-13T19:14:01.336Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of scale/translation-invariant
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is ``(x1, y1, x2, y2)`` in pixels; ``(track_id, box)`` when motion
needs it.
12    - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13    index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14    - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15    - ``court_polygon`` is normalised 0-1 ``(x, y), ...`` or ``None`` (None ? no mask,
every
16    detection counts ? the player-count-only mode).
17    - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33        if polygon is None or len(polygon) < 3:
34            return False
35        x, y = point
36        inside = False
37        n = len(polygon)

```

```

38     j = n - 1
39     for i in range(n):
40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51 def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52     """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53     the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54     if frame_width <= 0 or frame_height <= 0:
55         return (0.0, 0.0)
56     x1, y1, x2, y2 = box
57     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60 def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61     if frame_width <= 0 or frame_height <= 0:
62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67 def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                    court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69     """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70     True (count everyone) when it is None."""
71     if court_polygon is None:
72         return True
73     return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76 def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                    frame_height: float,
78                    court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79     """Per-frame count of in-court persons (one entry per sampled frame)."""
80     return [
81         sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
82         for _frame_idx, dets in sorted(per_frame.items())
83     ]
84
85
86 def _median(vals: Sequence[float]) -> float:
87     if not vals:
88         return 0.0
89     s = sorted(vals)
90     n = len(s)
91     return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94 def players_in_court(counts: Sequence[int]) -> float:
95     """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96     return _median(list(counts))
97
98

```

```

99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125
126     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                         frame_height: float, net: Optional[Net],
128                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133         0.0.
134
135         """
136         if net is None or not per_frame:
137             return 0.0
138         axis, pos = net
139         both = 0
140         for _frame_idx, dets in per_frame.items():
141             near, far = False, False
142             for _tid, box in dets:
143                 if not person_in_court(box, frame_width, frame_height, court_polygon):
144                     continue
145                 fx, fy = _foot_norm(box, frame_width, frame_height)
146                 coord = fx if axis == "x" else fy
147                 if coord < pos:
148                     near = True
149                 else:
150                     far = True
151             if near and far:
152                 both += 1
153         return both / len(per_frame)
154
155
156     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
157                        pad_frac: float) -> bool:
158         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
159         width/height (the 'adjacent / in-hand' tolerance)?"""
160         x1, y1, x2, y2 = box
161         px = pad_frac * frame_width
162         py = pad_frac * frame_height

```

```

161         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                                     frame_width: float, frame_height: float,
166                                     pad_frac: float = 0.03) -> float:
167         """Fraction of (visible shuttle frames that were also person-sampled) in which the
168         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
169         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
170         """
171         if not per_frame or frame_width <= 0 or frame_height <= 0:
172             return 0.0
173         aligned = 0
174         colocated = 0
175         for p in shuttle_points or []:
176             if not getattr(p, "visible", False):
177                 continue
178             dets = per_frame.get(int(p.frame))
179             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
180                 continue
181             aligned += 1
182             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183                     for _tid, box in dets):
184                 colocated += 1
185         return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                     frame_width: float, frame_height: float,
198                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                     net: Optional[Net] = None,
200                                     min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212

```

7. assistant (2026-06-13T19:14:04.677Z)

Now let me look at the WASB runner's windowing to confirm windows carry `start`/`end` in seconds and the frame?seconds conversion is identical, and check the `detections\_per\_frame` method to understand how `start\_f`/`end\_f` are used.

8. user (2026-06-13T19:14:06.362Z)

```
256: def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
257-                             frame_skip: int = 3) -> Dict[str, Any]:
258-     """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
video.
259-
260-     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
261-     ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards
the
262-     per-frame boxes), this returns the raw per-frame detections so the fusion features
can
263-     align them with the shuttle trajectory. It reads the source by **absolute frame
index**
264-     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
match
265-     the WASB trajectory's ``.frame`` exactly ? that alignment is what
266-     ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
267-
268-     Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
269-     "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
270-     accuracy varies by codec (it lands near the requested keyframe then decodes
forward);
271-     treat the indices as ±a few frames on real footage ? fine for window-level features.
272-     """
273-     self.lazy_load_model()
274-     cap = cv2.VideoCapture(video_path)
275-     if not cap.isOpened():
276-         logger.error(f"Could not open {video_path}")
277-         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
278-
279-     fps = cap.get(cv2.CAP_PROP_FPS)
280-     if fps <= 0:
281-         fps = 30.0
282-     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
283-     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
284-     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
285-     step = max(frame_skip, 1)
286-     tracker = self.make_tracker(fps / step)
287-
288-     start_frame = max(0, int(start_frame))
289-     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
290-     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
291-     frame_idx = start_frame
292-     while frame_idx <= end_frame:
293-         ret, frame = cap.read()
294-         if not ret:
295-             break
296-         if (frame_idx - start_frame) % step == 0:
297-             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
298-             frame_idx += 1
299-     cap.release()
300-     return {"frames": frames, "fps": fps,
301-            "frame_width": float(frame_width), "frame_height": float(frame_height)}
```

9. assistant (2026-06-13T19:14:10.376Z)

Now let me confirm the WASB runner's `trajectory\_to\_action\_windows` produces the same seconds-based start/end (the finding references tracknet_runner.py:300-301, but the default substrate is WASB). Let me check both and verify the claim about `start = group[0][0]/fps`.

10. user (2026-06-13T19:14:12.145Z)

No matches found

11. user (2026-06-13T19:14:21.592Z)

```
17-from typing import Dict, Any, Optional, Tuple
18-
19-# Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20:# parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on TrackNetRunner.
21-from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22-from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin, wsl_tilde_quote
23-
24-
25-
26-
27-
28-
29-
30-
31-
32-
33-
34-
35-
36-
37-
38-
39-
40-
41-
42-
43-
44-
45-
46-
47-
48-
49-
50-
51-
52-
53-
54-
55-
56-
57-
58-
59-
60-
61-
62-
63-
64-
65-
66-
67-
68-
69-
70-
71-
72-
73-
74-
75-
76-
77-
78-
79-
80-
81-
82-
83-
84-
85-
86-
87-
88-
89-
90-
91-
92-
93-
94-
95-
96-
97-
98-
99-
100-
101-
102-
103-
104-
105-
106-
107-
108-
109-
110-
111-
112-
113-
114-
115-
116-
117-
118-
119-
120-
121-
122-
123-
124-
125-
126-
127-
128-
129-
130-
131-
132-
133-
134-
135-
136-
137-
138-
139-
140-
141-
142-
143-
144-
145-
146-
147-
148-
149-
150-
151-
152-
153-
154-
155-
156-
157-
158-
159-
160-
161-
162-
163-
164-
165-
166-
167-
168-
169-
170-
171-
172-
173-
174-
175-
176-
177-
178-
179-
180-
181-
182-
183-
184-
185-
186-
187-
188-
189-
190-
191-
192-
193-
194-
195-
196-
197-
198-
199-
200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-
214-
215-
216-
217-
218-
219-
220-
221-
222-
223-
224-
225-
226-
227-
228-
229-
230-
231-
232-
233-
234-
235-
236-
237-
238-
239-
240-
241-
242-
243-
244-
245-
246-
247-
248-
249-
250-
251-
252-
253-
254-
255-
256-
257-
258-
259-
260-
261-
262-
263-
264-
265-
266-
267-
268-
269-
270-
271-
272-
273-
274-
275-
276-
277-
278-
279-
280-
281-
282-
283-
284-
285-
286-
287-
288-
289-
290-
291-
292-
293-
294-
295-
296-
297-
298-
299-
300-
301-
302-
303-
304-
305-
306-
307-
308-
309-
310-
311-
312-
313-
314-
315-
316-
317-
318-
319-
320-
321-
322-
323-
324-
325-
326-
327-
328-
329-
330-
331-
332-
333-
334-
335-
336-
337-
338-
339-
340-
341-
342-
343-
344-
345-
346-
347-
348-
349-
350-
351-
352-
353-
354-
355-
356-
357-
358-
359-
360-
361-
362-
363-
364-
365-
366-
367-
368-
369-
370-
371-
372-
373-
374-
375-
376-
377-
378-
379-
380-
381-
382-
383-
384-
385-
386-
387-
388-
389-
390-
391-
392-
393-
394-
395-
396-
397-
398-
399-
400-
401-
402-
403-
404-
405-
406-
407-
408-
409-
410-
411-
412-
413-
414-
415-
416-
417-
418-
419-
420-
421-
422-
423-
424-
425-
426-
427-
428-
429-
430-
431-
432-
433-
434-
435-
436-
437-
438-
439-
440-
441-
442-
443-
444-
445-
446-
447-
448-
449-
450-
451-
452-
453-
454-
455-
456-
457-
458-
459-
460-
461-
462-
463-
464-
465-
466-
467-
468-
469-
470-
471-
472-
473-
474-
475-
476-
477-
478-
479-
480-
481-
482-
483-
484-
485-
486-
487-
488-
489-
490-
491-
492-
493-
494-
495-
496-
497-
498-
499-
500-
501-
502-
503-
504-
505-
506-
507-
508-
509-
510-
511-
512-
513-
514-
515-
516-
517-
518-
519-
520-
521-
522-
523-
524-
525-
526-
527-
528-
529-
530-
531-
532-
533-
534-
535-
536-
537-
538-
539-
540-
541-
542-
543-
544-
545-
546-
547-
548-
549-
550-
551-
552-
553-
554-
555-
556-
557-
558-
559-
560-
561-
562-
563-
564-
565-
566-
567-
568-
569-
570-
571-
572-
573-
574-
575-
576-
577-
578-
579-
580-
581-
582-
583-
584-
585-
586-
587-
588-
589-
590-
591-
592-
593-
594-
595-
596-
597-
598-
599-
600-
601-
602-
603-
604-
605-
606-
607-
608-
609-
610-
611-
612-
613-
614-
615-
616-
617-
618-
619-
620-
621-
622-
623-
624-
625-
626-
627-
628-
629-
630-
631-
632-
633-
634-
635-
636-
637-
638-
639-
640-
641-
642-
643-
644-
645-
646-
647-
648-
649-
650-
651-
652-
653-
654-
655-
656-
657-
658-
659-
660-
661-
662-
663-
664-
665-
666-
667-
668-
669-
670-
671-
672-
673-
674-
675-
676-
677-
678-
679-
680-
681-
682-
683-
684-
685-
686-
687-
688-
689-
690-
691-
692-
693-
694-
695-
696-
697-
698-
699-
700-
701-
702-
703-
704-
705-
706-
707-
708-
709-
710-
711-
712-
713-
714-
715-
716-
717-
718-
719-
720-
721-
722-
723-
724-
725-
726-
727-
728-
729-
730-
731-
732-
733-
734-
735-
736-
737-
738-
739-
740-
741-
742-
743-
744-
745-
746-
747-
748-
749-
750-
751-
752-
753-
754-
755-
756-
757-
758-
759-
760-
761-
762-
763-
764-
765-
766-
767-
768-
769-
770-
771-
772-
773-
774-
775-
776-
777-
778-
779-
780-
781-
782-
783-
784-
785-
786-
787-
788-
789-
790-
791-
792-
793-
794-
795-
796-
797-
798-
799-
800-
801-
802-
803-
804-
805-
806-
807-
808-
809-
810-
811-
812-
813-
814-
815-
816-
817-
818-
819-
820-
821-
822-
823-
824-
825-
826-
827-
828-
829-
830-
831-
832-
833-
834-
835-
836-
837-
838-
839-
840-
841-
842-
843-
844-
845-
846-
847-
848-
849-
850-
851-
852-
853-
854-
855-
856-
857-
858-
859-
860-
861-
862-
863-
864-
865-
866-
867-
868-
869-
870-
871-
872-
873-
874-
875-
876-
877-
878-
879-
880-
881-
882-
883-
884-
885-
886-
887-
888-
889-
890-
891-
892-
893-
894-
895-
896-
897-
898-
899-
900-
901-
902-
903-
904-
905-
906-
907-
908-
909-
910-
911-
912-
913-
914-
915-
916-
917-
918-
919-
920-
921-
922-
923-
924-
925-
926-
927-
928-
929-
930-
931-
932-
933-
934-
935-
936-
937-
938-
939-
940-
941-
942-
943-
944-
945-
946-
947-
948-
949-
950-
951-
952-
953-
954-
955-
956-
957-
958-
959-
960-
961-
962-
963-
964-
965-
966-
967-
968-
969-
970-
971-
972-
973-
974-
975-
976-
977-
978-
979-
980-
981-
982-
983-
984-
985-
986-
987-
988-
989-
990-
991-
992-
993-
994-
995-
996-
997-
998-
999-
1000-
1001-
1002-
1003-
1004-
1005-
1006-
1007-
1008-
1009-
1010-
1011-
1012-
1013-
1014-
1015-
1016-
1017-
1018-
1019-
1020-
1021-
1022-
1023-
1024-
1025-
1026-
1027-
1028-
1029-
1030-
1031-
1032-
1033-
1034-
1035-
1036-
1037-
1038-
1039-
1040-
1041-
1042-
1043-
1044-
1045-
1046-
1047-
1048-
1049-
1050-
1051-
1052-
1053-
1054-
1055-
1056-
1057-
1058-
1059-
1060-
1061-
1062-
1063-
1064-
1065-
1066-
1067-
1068-
1069-
1070-
1071-
1072-
1073-
1074-
1075-
1076-
1077-
1078-
1079-
1080-
1081-
1082-
1083-
1084-
1085-
1086-
1087-
1088-
1089-
1090-
1091-
1092-
1093-
1094-
1095-
1096-
1097-
1098-
1099-
1100-
1101-
1102-
1103-
1104-
1105-
1106-
1107-
1108-
1109-
1110-
1111-
1112-
1113-
1114-
1115-
1116-
1117-
1118-
1119-
1120-
1121-
1122-
1123-
1124-
1125-
1126-
1127-
1128-
1129-
1130-
1131-
1132-
1133-
1134-
1135-
1136-
1137-
1138-
1139-
1140-
1141-
1142-
1143-
1144-
1145-
1146-
1147-
1148-
1149-
1150-
1151-
1152-
1153-
1154-
1155-
1156-
1157-
1158-
1159-
1160-
1161-
1162-
1163-
1164-
1165-
1166-
1167-
1168-
1169-
1170-
1171-
1172-
1173-
1174-
1175-
1176-
1177-
1178-
1179-
1180-
1181-
1182-
1183-
1184-
1185-
1186-
1187-
1188-
1189-
1190-
1191-
1192-
1193-
1194-
1195-
1196-
1197-
1198-
1199-
1200-
1201-
1202-
1203-
1204-
1205-
1206-
1207-
1208-
1209-
1210-
1211-
1212-
1213-
1214-
1215-
1216-
1217-
1218-
1219-
1220-
1221-
1222-
1223-
1224-
1225-
1226-
1227-
1228-
1229-
1230-
1231-
1232-
1233-
1234-
1235-
1236-
1237-
1238-
1239-
1240-
1241-
1242-
1243-
1244-
1245-
1246-
1247-
1248-
1249-
1250-
1251-
1252-
1253-
1254-
1255-
1256-
1257-
1258-
1259-
1260-
1261-
1262-
1263-
1264-
1265-
1266-
1267-
1268-
1269-
1270-
1271-
1272-
1273-
1274-
1275-
1276-
1277-
1278-
1279-
1280-
1281-
1282-
1283-
1284-
1285-
1286-
1287-
1288-
1289-
1290-
1291-
1292-
1293-
1294-
1295-
1296-
1297-
1298-
1299-
1300-
1301-
1302-
1303-
1304-
1305-
1306-
1307-
1308-
1309-
1310-
1311-
1312-
1313-
1314-
1315-
1316-
1317-
1318-
1319-
1320-
1321-
1322-
1323-
1324-
1325-
1326-
1327-
1328-
1329-
1330-
1331-
1332-
1333-
1334-
1335-
1336-
1337-
1338-
1339-
1340-
1341-
1342-
1343-
1344-
1345-
1346-
1347-
1348-
1349-
1350-
1351-
1352-
1353-
1354-
1355-
1356-
1357-
1358-
1359-
1360-
1361-
1362-
1363-
1364-
1365-
1366-
1367-
1368-
1369-
1370-
1371-
1372-
1373-
1374-
1375-
1376-
1377-
1378-
1379-
1380-
1381-
1382-
1383-
1384-
1385-
1386-
1387-
1388-
1389-
1390-
1391-
1392-
1393-
1394-
1395-
1396-
1397-
1398-
1399-
1400-
1401-
1402-
1403-
1404-
1405-
1406-
1407-
1408-
1409-
1410-
1411-
1412-
1413-
1414-
1415-
1416-
1417-
1418-
1419-
1420-
1421-
1422-
1423-
1424-
1425-
1426-
1427-
1428-
1429-
1430-
1431-
1432-
1433-
1434-
1435-
1436-
1437-
1438-
1439-
1440-
1441-
1442-
1443-
1444-
1445-
1446-
1447-
1448-
1449-
1450-
1451-
1452-
1453-
1454-
1455-
1456-
1457-
1458-
1459-
1460-
1461-
1462-
1463-
1464-
1465-
1466-
1467-
1468-
1469-
1470-
1471-
1472-
1473-
1474-
1475-
1476-
1477-
1478-
1479-
1480-
1481-
1482-
1483-
1484-
1485-
1486-
1487-
1488-
1489-
1490-
1491-
1492-
1493-
1494-
1495-
1496-
1497-
1498-
1499-
1500-
1501-
1502-
1503-
1504-
1505-
1506-
1507-
1508-
1509-
1510-
1511-
1512-
1513-
1514-
1515-
1516-
1517-
1518-
1519-
1520-
1521-
1522-
1523-
1524-
1525-
1526-
1527-
1528-
1529-
1530-
1531-
1532-
1533-
1534-
1535-
1536-
1537-
1538-
1539-
1540-
1541-
1542-
1543-
1544-
1545-
1546-
1547-
1548-
1549-
1550-
1551-
1552-
1553-
1554-
1555-
1556-
1557-
1558-
1559-
1560-
1561-
1562-
1563-
1564-
1565-
1566-
1567-
1568-
1569-
1570-
1571-
1572-
1573-
1574-
1575-
1576-
1577-
1578-
1579-
1580-
1581-
1582-
1583-
1584-
1585-
1586-
1587-
1588-
1589-
1590-
1591-
1592-
1593-
1594-
1595-
1596-
1597-
1598-
1599-
1600-
1601-
1602-
1603-
1604-
1605-
1606-
1607-
1608-
1609-
1610-
1611-
1612-
1613-
1614-
1615-
1616-
1617-
1618-
1619-
1620-
1621-
1622-
1623-
1624-
1625-
1626-
1627-
1628-
1629-
1630-
1631-
1632-
1633-
1634-
1635-
1636-
1637-
1638-
1639-
1640-
1641-
1642-
1643-
1644-
1645-
1646-
1647-
1648-
1649-
1650-
1651-
1652-
1653-
1654-
1655-
1656-
1657-
1658-
1659-
1660-
1661-
1662-
1663-
1664-
1665-
1666-
1667-
1668-
1669-
1670-
1671-
1672-
1673-
1674-
1675-
1676-
1677-
1678-
1679-
1680-
1681-
1682-
1683-
1684-
1685-
1686-
1687-
1688-
1689-
1690-
1691-
1692-
1693-
1694-
1695-
1696-
1697-
1698-
1699-
1700-
1701-
1702-
1703-
1704-
1705-
1706-
1707-
1708-
1709-
1710-
1711-
1712-
1713-
1714-
1715-
1716-
1717-
1718-
1719-
1720-
1721-
1722-
1723-
1724-
1725-
1726-
1727-
1728-
1729-
1730-
1731-
1732-
1733-
1734-
1735-
1736-
1737-
1738-
1739-
1740-
1741-
1742-
1743-
1744-
1745-
1746-
1747-
1748-
1749-
1750-
1751-
1752-
1753-
1754-
1755-
1756-
1757-
1758-
1759-
1760-
1761-
1762-
1763-
1764-
1765-
1766-
1767-
1768-
1769-
1770-
1771-
1772-
1773-
1774-
1775-
1776-
1777-
1778-
1779-
1780-
1781-
1782-
1783-
1784-
1785-
1786-
1787-
1788-
1789-
1790-
1791-
1792-
1793-
1794-
1795-
1796-
1797-
1798-
1799-
1800-
1801-
1802-
1803-
1804-
1805-
1806-
1807-
1808-
1809-
1810-
1811-
1812-
1813-
1814-
1815-
1816-
1817-
1818-
1819-
1820-
1821-
1822-
1823-
1824-
1825-
1826-
1827-
1828-
1829-
1830-
1831-
1832-
1833-
1834-
1835-
1836-
1837-
1838-
1839-
1840-
1841-
1842-
1843-
1844-
1845-
1846-
1847-
1848-
1849-
1850-
1851-
1852-
1853-
1854-
1855-
1856-
1857-
1858-
1859-
1860-
1861-
1862-
1863-
1864-
1865-
1866-
1867-
1868-
1869-
1870-
1871-
1872-
1873-
1874-
1875-
1876-
1877-
1878-
1879-
1880-
1881-
1882-
1883-
1884-
1885-
1886-
1887-
1888-
1889-
1890-
1891-
1892-
1893-
1894-
1895-
1896-
1897-
1898-
1899-
1900-
1901-
1902-
1903-
1904-
1905-
1906-
1907-
1908-
1909-
1910-
1911-
1912-
1913-
1914-
1915-
1916-
1917-
1918-
1919-
1920-
1921-
1922-
1923-
1924-
1925-
1926-
1927-
1928-
1929-
1930-
1931-
1932-
1933-
1934-
1935-
1936-
1937-
1938-
1939-
1940-
1941-
1942-
1943-
1944-
1945-
1946-
1947-
1948-
1949-
1950-
1951-
1952-
1953-
1954-
1955-
1956-
1957-
1958-
1959-
1960-
1961-
1962-
1963-
1964-
1965-
1966-
1967-
1968-
1969-
1970-
1971-
1972-
1973-
1974-
1975-
1976-
1977-
1978-
1979-
1980-
1981-
1982-
1983-
1984-
1985-
1986-
1987-
1988-
1989-
1990-
1991-
1992-
1993-
1994-
1995-
1996-
1997-
1998-
1999-
2000-
2001-
2002-
2003-
2004-
2005-
2006-
2007-
2008-
2009-
2010-
2011-
2012-
2013-
2014-
2015-
2016-
2017-
2018-
2019-
2020-
2021-
2022-
2023-
2024-
2025-
2026-
2027-
2028-
2029-
2030-
2031-
2032-
2033-
2034-
2035-
2036-
2037-
2038-
2039-
2040-
2041-
2042-
2043-
2044-
2045-
2046-
2047-
2048-
2049-
2050-
2051-
2052-
2053-
2054-
2055-
2056-
2057-
2058-
2059-
2060-
2061-
2062-
2063-
2064-
2065-
2066-
2067-
2068-
2069-
2070-
2071-
2072-
2073-
2074-
2075-
2076-
2077-
2078-
2079-
2080-
2081-
2082-
2083-
2084-
2085-
2086-
2087-
2088-
2089-
2090-
2091-
2092-
2093-
2094-
2095-
2096-
2097-
2098-
2099-
2100-
2101-
2102-
2103-
2104-
2105-
2106-
2107-
2108-
2109-
2110-
2111-
2112-
2113-
2114-
2115-
2116-
2117-
2118-
2119-
2120-
2121-
2122-
2123-
2124-
2125-
2126-
2127-
2128-
2129-
2130-
2131-
2132-
2133-
2134-
2135-
2136-
2137-
2138-
2139-
2140-
2141-
2142-
2143-
2144-
2145-
2146-
2147-
2148-
2149-
2150-
2151-
2152-
2153-
2154-
2155-
2156-
2157-
2158-
2159-
2160-
2161-
2162-
2163-
2164-
2165-
2166-
2167-
2168-
2169-
2170-
2171-
2172-
2173-
2174-
2175-
2176-
2177-
2178-
2179-
2180-
2181-
2182-
2183-
2184-
2185-
2186-
2187-
2188-
2189-
2190-
2191-
2192-
2193-
2194-
2195-
2196-
2197-
2198-
2199-
2200-
2201-
2202-
2203-
2204-
2205-
2206-
2207-
2208-
2209-
2210-
2211-
2212-
2213-
2214-
2215-
2216-
2217-
2218-
2219-
2220-
2221-
2222-
2223-
2224-
2225-
2226-
2227-
2228-
2229-
223
```

```

frames={cw['active_frame_count']} "
        f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

+     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+     # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+     # persistence and the handoff gate below.
+     window_stats = [
+         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+         for cw in all_candidate_windows
+     ]
+
+     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
+     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
+     if store_candidates:
+         for i, cw in enumerate(all_candidate_windows):
+             stats = {
+                 "peak_velocity": cw["peak_velocity"],
+                 "mean_velocity": cw["mean_velocity"],
+                 "active_frame_count": cw["active_frame_count"],
+                 "track_id_count": cw["track_id_count"],
+             }
+             candidate_ids[i] = self.db.add_candidate_segment(
+                 video_id, detector=self.name,
+                 start_time=cw["start"], end_time=cw["end"],
+                 chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
+                 stats=stats, detection_params=detection_params,
+                 stats=window_stats[i], detection_params=detection_params,
+             )
+
+     if not all_candidate_windows:
+         return []

+     # --- Phase 3: AI provider handoff over padded candidate clips ---
+     # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+     # candidates above stay the full superset ? only the served play_segments are gated.
+     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
+     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
+     os.makedirs(handoff_dir, exist_ok=True)
+     logger.info("Phase 3: %s validation on %d candidate windows...",
+                 provider_name, len(all_candidate_windows))
+
+     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
+         w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
+
+         ai_segments: List[dict] = []
+         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
+             futures = [executor.submit(process_candidate_window, wi, cw)
+                         for wi, cw in enumerate(all_candidate_windows)]
+             futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                         for wi in handoff_indices]
+             for future in futures:
+                 ai_segments.extend(future.result())
+
+     try:

```

14. assistant (2026-06-13T19:14:37.696Z)

Now I have a clear picture. The window dict `cw` contains `start`/`end` in seconds (from `group[0][0]/fps + chunk\_start`), and `active\_frame\_count` but NOT the raw integer frame indices `group[0][0]/`group[-1][0]`. So the fusion code does round-trip `int(cw["start"] * fps)` to reconstruct frame bounds.

Let me empirically verify the core claim about `int(frame/fps*fps)` truncating to `frame-1`. Let me test the actual numbers cited in the finding.

15. user (2026-06-13T19:14:40.836Z)

```
fps=30.00000: mismatches=404 examples=[(123, 122), (245, 244), (246, 245)]
fps=29.97003: mismatches=477 examples=[(61, 60), (121, 120), (122, 121)]
fps=25.00000: mismatches=1145 examples=[(29, 28), (57, 56), (58, 57)]
```

16. assistant (2026-06-13T19:14:44.387Z)

The reproduction matches the finding's numbers exactly (404 @ 30, 477 @ 29.97, 1145 @ 25, and the cited example `123 -> 122`). Now let me verify `round()` actually fixes this and assess whether the consequences described are accurate.

17. user (2026-06-13T19:14:46.167Z)

```
fps=30.00000: round() mismatches=0
fps=29.97003: round() mismatches=0
fps=25.00000: round() mismatches=0
```

18. assistant (2026-06-13T19:14:55.853Z)

The proposed `round()` fix eliminates all mismatches. Now let me critically assess the severity and consequences. Let me verify the two claimed consequences against the actual code.

****Consequence 1 ? `detections\_per\_frame` seeded one frame early:**** In `person_detector.py:296`, `if (frame_idx - start_frame) % step == 0`. If `start_frame` is off by 1 (e.g. 122 instead of 123), the sampling phase shifts by one frame relative to the shuttle's actual `.frame` values. Let me verify whether this actually breaks alignment with the shuttle frames used in `shuttle_player_colocation`.

19. user (2026-06-13T19:14:58.212Z)

```
start_f=122: sampled frames [122, 125, 128, 131, 134, 137, 140, 143]
start_f=123: sampled frames [123, 126, 129, 132, 135, 138, 141, 144]
```

20. assistant (2026-06-13T19:15:14.815Z)

This confirms the finding's claim is precise. The off-by-one shifts the entire sampling phase. The shuttle trajectory `.frame` values are absolute integer frame indices (123, 124, 125...). With `start_f=122`, person frames sampled are {122,125,128,...} vs shuttle visible frames at e.g. {123,126,129,...}. The `shuttle_player_colocation` (`person_detector.py` keyed by `int(p.frame)`) requires `per_frame.get(int(p.frame))` to hit. A one-frame phase shift can reduce alignment hits ? though cv2 keyframe seek tolerance already perturbs this.

Let me verify the boundary-filter consequence (`shuttle_in` filter at line 105) and confirm the finding's framing that this is bounded/benign, then assess the final severity. The finding correctly self-rates this as ****low**** severity. Let me confirm the docstring already acknowledges cv2 seek tolerance dwarfs this, which is the key mitigating context.

The `person_detector.py:269-271` docstring explicitly states: "cv2 frame-seek accuracy varies by codec... treat the indices as $\pm$ a few frames on real footage ? fine for window-level features." This confirms the finding's own caveat that the cv2 tolerance dwarfs the 1-frame error.

I have now adversarially verified every claim:

- **Round-trip truncation is real and reproduces exactly**** ? 404/477/1145 mismatches at fps 30/29.97/25, example `123?122` matches.
- **The code genuinely does the round-trip**** ? `fusion_hybrid.py:97` `start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)`, and the window dict does NOT carry integer frame indices (only seconds + `active_frame_count`), so the round-trip is genuinely necessary as written.
- **`round()` fixes it**** ? 0 mismatches.

4. ****Consequences are accurately characterized**** ? sampling phase shift (line 98 ? ``person_detector.py:296``) and boundary filter (``fusion_hybrid.py:105``), both bounded.
5. ****Self-rated low severity is justified**** ? gated behind person cue (default OFF), and cv2 seek tolerance (acknowledged in the substrate docstring) already dwarfs a 1-frame error.

One refinement: the entire ``_person_features`` path only runs when ``cue_flags.person=True``, which the module docstring confirms is OFF by default. The ``int()`` truncation also appears in ``_select_for_handoff``? No ? that path uses ``net_crossings`` via ``rally_gate.gate_windows(points, [(cw["start"], cw["end"]]), fps, ...)``, which I should briefly confirm doesn't share this exact bug, but it's a separate code path not claimed by this finding. The finding is scoped precisely to lines 97 and 105 of the person path.

The finding holds against the code. It is a real, low-severity precision issue, not a false positive or nitpick fabrication ? the numbers, the mechanism, and the fix all check out.

21. user (2026-06-13T19:15:28.449Z)

Structured output provided successfully

22. assistant (2026-06-13T19:15:36.645Z)

I've verified the finding adversarially against the actual code and confirmed it is ****real**** at ****low**** severity.

Key verification points:

- ****Reproduced the exact numbers****: 404 mismatches at fps=30, 477 at 29.97, 1145 at 25.0, with example frame 123?122 ? all match the finding precisely.
- ****Confirmed the round-trip is genuinely necessary as written****: the window dict (``backend/pipeline/detectors/tracknet_runner.py:300-301``, reused by WASB) carries ``start`/`end`` in seconds only, not the integer frame indices, so ``fusion_hybrid.py:97`` must reconstruct via ``int(cw["start"] * fps)``.
- ****Confirmed the consequences****: the off-by-one shifts the person-sampling phase (``person_detector.py:296``) and can shift the boundary filter (``fusion_hybrid.py:105``) by one frame ? bounded, non-corrupting.
- ****Confirmed the fix****: ``round()`` yields 0 mismatches at all three frame rates.
- ****Confirmed low severity is justified****: the path is gated behind ``cue_flags.person`` (default OFF), and the substrate's own docstring (``person_detector.py:269-271``) notes cv2 keyframe-seek is already $\pm$ a few frames on real footage, which dwarfs this 1-frame error.

Not a false positive or nitpick fabrication ? the mechanism, measured numbers, file:line anchors, and proposed fix all hold against the code.